- Can be used in self-service, to keep the unit's autonomy and ability to react to change
- Enable units to experiment with variations, to help them react to change

In our woodworking example, this is exactly what we have. Every woodworker can go on as desired. No one is forcing the woodworkers to do anything different, but they have the option of using the tool-making station. The station has enabling tools such as measurement instruments, pencils, and drawing boards—everything needed to design, prototype, and build special-purpose tools. Wednesdays help the woodworkers widen their horizons and provide ideas to experiment with, in effect, helping the woodworkers to come up with good experiments to run.

Traditional platforms

The word *platform* makes some CIO/CTOs shiver because a lot of platforms do not support the idea of platform thinking. If your "platform" is a fancy deployment machine that requires every development team to have an expert who can use that fancy deployment machine, no team is actually "free to choose to use the platform." Hence, there is a boundary in adopting the platform. Such a platform will likely restrict the team because it locks them in. After all, if you have a Jenkins expert on every team, there is next to no incentive to switch to a 20% better platform that is not Jenkins.

The platforms that emerge out of platform thinking are lean and thin. These platforms are never launched as big and comprehensive pieces. Instead, they are built iteratively and carefully to enable teams and not limit their autonomy.

In fact, the larger the participant base becomes, the leaner the platform should become, in a sense. With more participants, the friction points grow, and we need to avoid friction at all costs; otherwise, we slow the flow, instead of enhancing it.

Next, let's apply platform thinking to our MVP platform.

Application to our platform